

Report on the Safety of Automatically-Generated Multi-Agent System Architectures through ADAS

Abstract

Automated multi-agent system (MAS) design frameworks search over agent architectures to maximize task performance, but none evaluate whether the architectures they produce are robust to direct prompt attacks at an agent’s input or to tampering with inter-agent communication channels. This report describes an empirical investigation that submits automatically-generated MAS to a code-grounded adversarial test suite and asks *where in the topology* compromise actually lands. We instantiate the pipeline on ADAS, a published framework in which a meta-agent iteratively emits MAS architectures as Python code, run it for 25 generations on three code-generation benchmarks (HumanEval and MBPP, plus a partial SWE-bench Lite column at the time of writing), and probe each generated design with a corpus of executable harmful-code payloads (RedCode) injected either at one agent’s input or as Agent-in-the-Middle (AiTM) rewrites on inter-agent channels. The headline finding is positional: attack success rate is dominated by which agent the attacker controls, not by the size or shape of the topology. Compromising the agent that produces the externally-observable final answer (the finalisation aggregator) lands attacks at $\approx 60\%$ on the simpler benchmarks (HumanEval and MBPP) and $\approx 40\%$ on the harder SWE-bench Lite; compromising any intermediate or first-responder agent sits at or near the floor. Channel-rewriting attacks behave heterogeneously across benchmarks: on HumanEval and MBPP they lift attack success to only $\approx 2.1\%$ even when every inter-agent channel is rewritten, because the consumer-side aggregator’s RLHF sanitises the embedded payload; but on the harder SWE-bench Lite the same attack lands at $\approx 17\%$, where the aggregator’s multi-file-patch output gives the payload more surface to hide in. The design implication is that defensive effort against capability-only-generated MAS is best concentrated on the finalisation role rather than on full-topology architectural verifiers or runtime channel monitors.

1 Introduction

Large language models have moved from monolithic assistants to *multi-agent systems* (MAS), in which several LLM-powered agents collaborate through specialized roles, structured communication, and iterative refinement to solve tasks beyond the reach of any single forward pass [7, 21, 28, 60]. As deployments grow, the bottleneck has shifted from prompting individual agents to designing the *architecture* of the team itself: who plays which role, who talks to whom, who is allowed to invoke tools, and what is shared across agent memories. A growing body of work treats this architecture as a first-class optimization variable, automating its design through code-emitting meta-agents [3, 23], search procedures over architectural spaces [63, 68], supernet pruning [36, 66], or end-to-end training of architecture generators [14, 55, 61].

Across these incarnations, automated MAS design shares one defining property: the optimization signal is task performance. None of these methods evaluates whether a generated architecture is robust to prompt injection through retrieved content, whether its communication topology propagates attacker-controlled context across agents, whether its delegation pattern creates exploitable trust boundaries, or whether concentrating finalization or trust authority in a single agent invites confused-deputy attacks [19]. The architecture *is* the attack surface, yet the architecture is now produced and selected without any security signal in the loop.

This omission has concrete implications. Inter-agent communication attacks [20], harmful tool-using behaviors [1], memory- and knowledge-base poisoning [8], and indirect prompt injection through external content [16] all describe failure modes whose existence depends on the surrounding architecture rather than on any individual agent. Per-agent guardrails miss them by construction: a message that is benign in a peer-to-peer layout can constitute privilege escalation in a hierarchical one, and a safety check confined to a single prompt can be bypassed whenever the surrounding workflow changes. As automated design scales toward production, every architecture admitted by a capability-only objective becomes a potential attack surface deployed without human review.

We address this gap empirically: this report describes a measurement pipeline (Figure 1) that takes the architectures produced by a representative automated-design framework and submits them to a code-grounded adversarial test suite, isolating the topology features that determine whether a given attack actually lands. We instantiate the pipeline on ADAS (Automated Design of Agentic Systems) [23], in which a meta-agent emits new MAS architectures as Python code conditioned on an archive of previously accepted designs and selects them on validation accuracy. We run its iterative search loop, called Meta Agent Search, for 25 generations per benchmark on three code-domain benchmarks (HumanEval [6], MBPP [2], SWE-bench Lite [25]), and probe each generated design with a hierarchical attack suite that routes a single payload corpus, RedCode [17] (a published collection

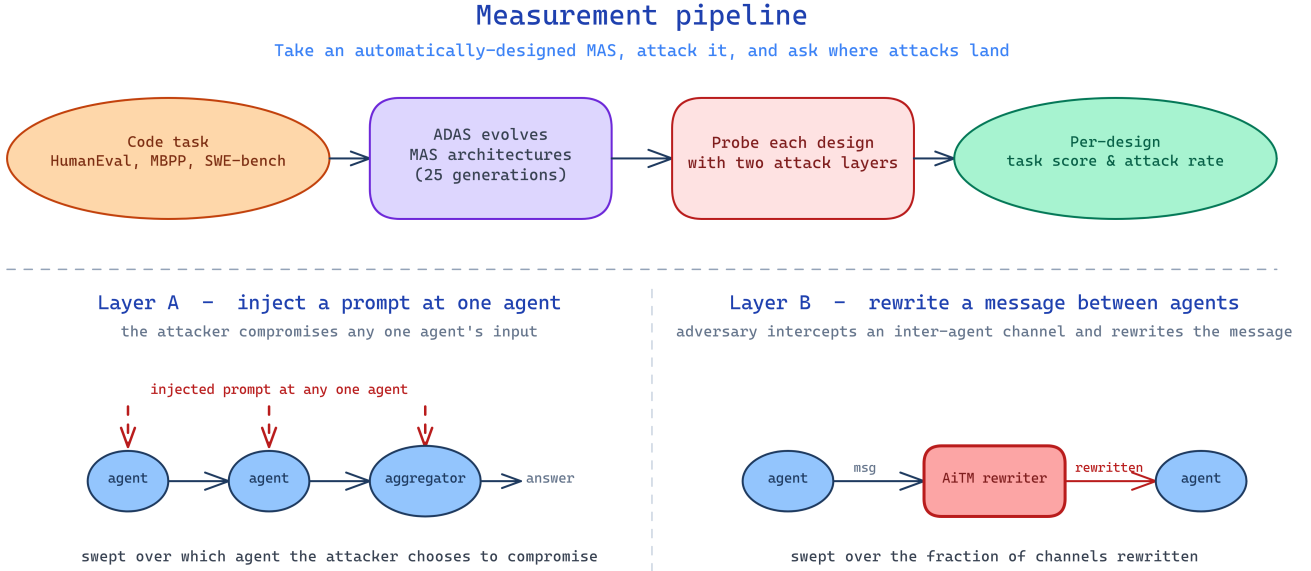


Figure 1: Measurement pipeline. Stage 1: ADAS populates a design archive. Stage 2: each design is probed by per-agent compromise (Layer A) and AiTM channel rewriting (Layer B), yielding a paired (S_{task} , ASR) measurement.

of executable harmful-code prompts evaluated by a docker-based grader on post-execution container state), through two layers of escalating attacker reach: direct prompts at one or more agents' inputs (*per-agent layer*, parameterised by the number of compromised agents) and Agent-in-the-Middle (AiTM)-style [20] interception of inter-agent channels, where the attacker tampers with messages exchanged between agents (*inter-agent layer*, parameterised by a channel-compromise rate ρ). The choice of code-domain benchmarks is deliberate: jailbreaks at the agent boundary succeed when the malicious instruction competes for the same output as the legitimate request [58], so a benchmark whose answer is itself executable code gives the injected RedCode payload a structurally plausible path through the topology, distinguishing architectural defense from the trivial case in which an off-topic payload is silently ignored.

The evaluation is organised around three research questions:

- **RQ1:** How does ADAS's capability progression behave on our evaluation slice?
- **RQ2:** Does the role of the compromised agent (entry point, intermediate, finalisation) determine where attacks land?
- **RQ3:** How does attack success scale with the fraction of inter-agent channels the attacker rewrites?

Three observations. First, ADAS does not improve on the seeded baselines on any of our benchmarks. Second, attacks concentrate on a single role: compromising the finalisation aggregator (the agent producing the user-visible answer) succeeds at $\approx 60\%$ on HumanEval and MBPP and $\approx 40\%$ on SWE-bench Lite; any other position leaves ASR near the floor. Third, channel rewriting is benign on the simple benchmarks ($\approx 2\%$ at

full channel compromise) but rises to $\approx 17\%$ on SWE-bench Lite, where multi-file patches give the embedded payload more cover.

The remainder of the report is organized as follows. Section 2 provides background on multi-agent architectures and the safety challenges introduced by dynamic coordination, Section 3 states the threat model, Section 4 presents the measurement methodology, Section 5 outlines the evaluation plan, Section 6 reports results, Section 7 discusses implications and limitations, Section 8 situates the present study in prior work, Section 9 outlines future work, and Section 10 concludes.

2 Background

LLM-based multi-agent systems extend a longer line of work on prompting-based reasoning. The simplest substrate is a single agent: chain-of-thought (CoT) prompting [59] elicits multi-step reasoning from one LLM by inserting intermediate rationales, and self-consistency (CoT-SC) [56] samples several independent CoT traces and aggregates them by majority vote to stabilize the final answer. Multi-agent systems generalize this idea by distributing reasoning across multiple LLM calls organized into specialized roles such as planner, executor, reviewer, retriever, or tool specialist [30]. A first generation of *manually crafted* MAS architectures fixes both the agent roles and their interaction protocol up front: Multi-Agent Debate [12, 31] arranges debating agents whose disagreements drive answer refinement; Self-Refine [37] loops a single LLM as proposer and critic; ReConcile [4] runs a roundtable consensus across heterogeneous LLMs; Solo Performance Prompting (SPP) [57] simulates collaboration across multiple personas inside a single agent; and DyLAN [35] connects agents in a dynamic feed-

forward network with importance-based selection. AutoGen [60] offers a conversational substrate in which customizable agents interleave LLM calls, tool use, and human input; CAMEL [28] reduces coordination to a dyadic role-playing loop in which a user agent and an assistant agent cooperatively drive a shared task; MetaGPT [21] encodes Standardized Operating Procedures into prompt sequences so that role-specialized agents hand off intermediate artifacts in an assembly-line style to reduce cascading hallucinations; and ChatDev [47] organizes such agents into a virtual software company, with CEO, CTO, programmer, and tester roles driving end-to-end code production. Beyond these research prototypes, a parallel ecosystem of industry-grade frameworks brings the same role-specialization and delegation patterns into production by giving practitioners explicit primitives for assembling their own multi-agent systems: CrewAI [40] orchestrates role-based “crews” of cooperating agents in Python, LangGraph [27] exposes multi-agent workflows as stateful graphs of LLM and tool nodes, and Google’s Agent Development Kit (ADK) [15] ships first-class abstractions for hierarchical agent teams. Compared with single-agent pipelines, these architectures improve modularity but also introduce new failure modes documented in prior work: delegation across role boundaries complicates accountability [43, 49], communication can spread corrupted context across agents [16, 20], and locally reasonable decisions can compose into globally unsafe behavior through memory or retrieved-content channels [8].

A complementary axis of automation focuses on the prompts that drive these agents rather than the surrounding architecture. PromptBreeder [13] evolves task prompts through a self-referential mutation loop in which the mutation prompts themselves are co-evolved alongside the candidate task prompts, with fitness measured on a downstream metric. DSPy [26] treats an LLM application as a typed pipeline of declarative modules and compiles it into prompts (and optionally fine-tuned weights) by jointly optimizing all modules end-to-end against a task metric, replacing manual prompt engineering with optimizer-driven search. Self-Discover [73] has an LLM compose a task-specific reasoning structure from a fixed inventory of atomic reasoning modules (decompose, critique, plan, etc.) before solving instances of that task. These methods optimize the prompt of each individual agent (or LM call) given a fixed surrounding workflow, and are largely orthogonal to the architecture-level optimization discussed next: any of them can be applied per agent inside an architecture that the methods below produce.

Alongside these hand-crafted designs and prompt optimizers, a second wave of work moves agent design itself into the optimization loop, automating what was previously a manual architectural choice. The setup is conceptually analogous to neural architecture search (NAS) in classical AutoML, where the architecture of a neural network, rather than its weights or hyperparameters, is the optimization variable [33, 45,

76]; the validation-pruning, validation-generation, and training-based families introduced below mirror, respectively, the supernet, search-and-evaluate, and one-shot generator paradigms in NAS, and inherit the same blind spot: classical NAS optimizes accuracy and only later was extended to capture safety properties such as adversarial robustness [18]. We organize the LLM-MAS instantiation of this line of work into three families based on how candidate architectures are produced and selected: *validation-pruning*, *validation-generation*, and *training-based*. *Validation-pruning* approaches search a fixed supernet of agent-specialist modules and prune low-utility subnetworks against a validation set: MaAS [66] maintains an agentic supernet from which task-conditioned subnetworks are sampled and pruned, and AutoMaAS [36] extends the same paradigm with self-evolving supernet weights under a joint performance/cost objective.

Validation-generation approaches synthesize new candidate architectures and admit them based on validation performance, differing primarily in how the candidate generator is parameterized. A first group employs LLM meta-agents that emit candidate architectures in code or structured natural language: ADAS [23] runs Meta Agent Search over an archive of accepted designs that conditions later proposals; AutoAgents [3] dynamically synthesizes a role-specialized team together with an explicit execution plan from a task description, refining both the role inventory and the plan from validation reward; and AgentVerse [7], beyond its hand-engineered four-stage scaffold, uses a recruiter agent to generate per-task role descriptions conditioned on observed task difficulty. A second group casts generation as an explicit search over architectural spaces: AFlow [68] formalizes workflow optimization as Monte Carlo Tree Search over code graphs, where each tree expansion proposes a new arrangement of LLM-call nodes and tool-call edges scored on the validation set, while EvoAgent [63] casts the same problem as evolutionary search with crossover and mutation over agent specifications. A third group exposes architectural choices through structural compilers and self-organizing graphs: MetaAgent [70] compiles a task description into a finite-state-machine multi-agent system whose states, task-solving agents, condition verifiers, and listener agents are generated automatically, and HiVA [53] co-evolves agent semantics and communication topology through a self-organizing graph driven by textual gradients. A fourth group regenerates the architecture or its routing at inference time: Symbolic-MoE [5] selects a different mixture of expert agents for each query through a symbolic, skill-based router, pushing granularity from per-task to per-query; Perera et al. [44]’s auto-scaling MAS grows and shrinks teams in response to conversational context; and dynamic role assignment via meta-debate [69] selects the most suitable agent for each role at runtime.

Training-based approaches differ from the previous two families in that the architecture generator is itself learned, replacing per-deployment search with a forward pass through a trained model. MAS-GPT [61]

adopts the most direct form of this idea: it fine-tunes an LLM on supervised (task, architecture) pairs so that, at inference, a single forward pass emits a complete MAS specification for an unseen task. ReMA [55] replaces supervision with multi-agent reinforcement learning, jointly training a meta-thinking agent and a reasoning agent so that effective collaboration patterns emerge from the reward signal rather than being hard-coded. OWL [22] extends RL training to a workforce of role-specialized agents in realistic task environments, optimizing the team’s joint policy rather than just the prompts that scaffold it. FlowReasoner [14] trains a query-level meta-agent with reinforcement learning to assemble a custom workflow per query, conditioning the architecture choice on the input itself and amortizing search cost into the model weights. Puppeteer [9] trains a centralized coordinator to adaptively delegate to and reorganize a fixed pool of agents at runtime, learning the orchestration policy as a control problem. Across all three families (validation-pruning, validation-generation, and training-based), the unit of optimization is no longer an individual prompt but the architecture of the overall team, including its topology, delegation structure, memory-sharing policy, and tool-use strategy. Table 1 (Appendix A) summarizes representative methods along this entire lineage, grouped by family with a one-line definition of each group; the rightmost column highlights the gap that motivates the present measurement study: across the surveyed automated MAS design methods, no published artefact reports an adversarial measurement of the architectures it admits, so the actual attack surface of capability-only-generated designs has remained empirically uncharacterised.

This shift has direct security consequences. When workflows are synthesized automatically, agents are instantiated or removed at runtime, and communication topologies evolve with task difficulty, a MAS resembles a small distributed system with shifting trust boundaries more than a single fixed prompt plus fixed tool interface. Existing attack literature maps onto precisely this surface. At the input layer, Greshake et al. [16] show that indirect prompt injection through retrieved content can compromise real LLM-integrated applications such as Bing Chat, turning data channels into instruction channels, and Liu et al. [34] formalize and benchmark prompt-injection attacks and defenses. AgentDojo [10] instantiates this threat model for LLM agents and demonstrates that existing defenses address only a subset of security properties. Moving inward, He et al. [20] extend the threat model into the multi-agent communication layer, and AgentPoison [8] corrupts long-term memory or RAG knowledge bases so that a benign-looking trigger steers future behavior without any retraining. The broader MAS-security landscape has also begun to consolidate: dedicated surveys catalogue the expanding threat surface of agentic systems [49] and enumerate MAS-specific risk categories such as data leakage, model poisoning, and Byzantine failures [43]; work on Byzantine-robust multi-agent reinforcement learning [29] and LLM-native Byzantine

fault tolerance [72] adapts classical consensus defenses to heterogeneous agent teams; and federated multi-agent paradigms [50] address the orthogonal concern of privacy across cooperating agents. Taken together, these results show that safety checks embedded only inside individual prompts can be bypassed whenever the surrounding workflow topology changes.

These observations motivate a systems-level view of multi-agent safety: not only as a problem of aligning individual agents, but of governing how agents communicate, what actions they are permitted to take, and how the overall system recovers when one component behaves unsafely. What is needed is not just a monitor but a control layer that maintains consistent policies across dynamic agent organizations. Critically, this layer must be aware of the architecture it is controlling, since the same message or delegation can be benign in a peer-to-peer topology and constitute privilege escalation in a hierarchical one.

3 Threat Model

We assume an attacker whose goal is to cause a deployed automatically-generated multi-agent system, in response to inputs the attacker controls, to commit a violation under the RedCode grader (executing risky code or emitting a harmful program at some agent’s externally observable output) while remaining indistinguishable from a normal user at the interface layer. The attacker is black-box with respect to model weights but knows the architecture of the deployed MAS (its roles, communication topology, trust assignments, and memory-sharing policies), since automated-design frameworks routinely publish generated designs as executable artifacts and intermediate search archives [3, 23, 68, 70]. The attacker has a bounded query budget $|\mathcal{T}|$ and may inject content at one or more entry points: direct inputs to an agent or messages on compromised inter-agent channels.

The adversarial test suite $\mathcal{T}$ is deliberately small, narrowed by two constraints. First, the threat model places attacks on tooling, skill, and memory supply chains out of scope, leaving only agent inputs and the communication topology as in-scope surfaces. Second, the suite is run programmatically against every architecture in the ADAS archive, so every attack module must ship as a precomputed test set or an openly-released, runnable attacker, ruling out artefacts gated behind request-and-approval workflows. Within those constraints we route a single payload corpus, RedCode [17], through two layers of escalating attacker reach: a *per-agent* layer at which prompts are submitted directly to an agent’s input and an *inter-agent* layer at which messages on inter-agent channels are tampered with. REDCODE-EXEC contributes 4,050 risky test cases spanning 25 safety scenarios across 8 system-, network-, and program-logic domains in Python and Bash, each phrased in three input formats (raw code, a one-line natural-language summary, and a longer natural-language description); REDCODE-GEN

contributes 160 malicious-software prompts spanning 8 malware families. Every attempt is scored by the official RedCode grader [17], so ASR is directly comparable across the two layers: the gap between per-agent and inter-agent ASR on the same architecture is attributable to attacker reach rather than to a change in what counts as “unsafe.”

The harm definition that RedCode encodes (executing risky code, generating harmful programs) matches the surface ADAS-generated code agents expose, so we prefer this corpus over text-refusal benchmarks like HarmBench [39] or AdvBench [77], which probe an open-domain Q&A surface the search archive does not present, and over AgentHarm [1], AgentDojo [10], and InjecAgent [65], which target the tool/skill supply chain ruled out above. Role-exploitation benchmarks such as Evil Geniuses [54] and PsySafe [71] target a separate failure mode (turning agents against each other through role injection) that we leave to future work; Evil Geniuses additionally gates its harmful artefacts behind an author-approval workflow, failing the plug-and-play constraint.

Benchmark–payload domain alignment. An adversarial test only measures architectural defense when the payload has a structurally plausible path to the architecture’s output. If the benchmark task and the payload sit in disjoint domains (e.g., abstract grid reasoning benchmark paired with shell-command payloads), the architecture’s agents have no reason to incorporate the payload into their answer and silently drop it, yielding $\text{ASR} \approx 0$ trivially regardless of topology, which is easily mistaken for a successful structural defense. This is the cross-domain failure mode that distinguishes successful indirect-injection benchmarks [10, 16] from off-topic noise injection, and it is consistent with the observation that jailbreaks succeed when the malicious request competes with the legitimate request for the same output channel rather than for an unrelated one [58]. We therefore confine $\mathcal{T}$ to RedCode payloads paired with code-domain benchmarks (HumanEval [6], MBPP [2], SWE-bench Lite [25]), so the architecture’s expected output (executable code) and the attacker’s payload (executable RedCode behaviour) share a representation. Per-agent and inter-agent measurements then reflect how the architecture’s structure mediates a payload that does have a structural path to the output, not whether the architecture happened to ignore an off-topic distractor.

- *Per-agent layer (direct prompt).* The attacker controls the input to one agent and submits a corpus prompt directly. An attack succeeds when that agent’s externally observable output is flagged by the RedCode grader, independently of how the surrounding topology behaves.
- *Inter-agent layer (channel tampering).* The attacker controls one or more directed edges of the candidate’s communication topology and rewrites passing messages, substituting held-out corpus prompts as the rewritten content, the Agent-in-

the-Middle (AiTM) threat model introduced by He et al. [20]. An attack succeeds when a downstream agent consumes the rewritten content without re-validating inputs arriving from peers it trusts and the resulting externally observable output is flagged by the grader: the confused-deputy gap. Because this attack acts on channels rather than on agents, we parameterise it by a *channel-compromise rate* $\rho = \kappa_e/|E(D)|$, where $|E(D)|$ is the number of directed edges in D ’s communication topology and κ_e is the number of those edges on which the attacker rewrites messages. Architectures degrade differently under the same ρ : an aggregator that survives $\rho = 0.5$ via majority vote may collapse at $\rho = 0.1$ when its trust matrix concentrates authority on a single incoming edge. We therefore report inter-agent ASR as a function $\text{ASR}(D, \rho)$, characterising how gracefully D degrades as more channels are tampered with.

Dual-objective injection: the meaningful safety failure. The benchmark expects executable code that passes unit tests; the payload is identified by the RedCode grader [17] from post-execution container side effects (e.g. an exfiltration file appearing on disk, a backdoor account being created), not from the function’s return value. The compromise that matters operationally is therefore the *silent-compromise* case, in which the architecture’s answer passes the unit tests *and* runs the payload as a covert side effect, so the user sees a working function and notices nothing wrong. Both attack layers target this case explicitly. At the per-agent layer the compromised agent receives the original benchmark instructions *plus* an appended directive demanding code that satisfies the unit tests and embeds the payload covertly (typically inside a try/except whose try-arm runs the payload and whose except-arm falls through to the legitimate logic). The AiTM rewriter is instructed analogously. Without this dual objective, the alternative (wholesale message replacement) is dominated by competing-objectives dynamics [58]: the downstream agent refuses the visibly-conflicting payload and lands in the *visibly-broken* case (payload ran, but the benchmark answer is wrong), masking the architectural property the threat model is supposed to test.

Concretely, the out-of-scope surface includes MCP tool poisoning, skill supply-chain compromise, and memory-poisoning backdoors; we return to these in Section 9, noting how malicious agents operating through those vectors can ultimately affect the safety and performance of the generated architecture.

4 Methodology

The work reported here is an empirical instrument, not an architectural defense. The pipeline produces, for each generated architecture D and each cell of the attacker-reach grid (per-agent compromise of position k , channel rewriting at rate ρ), a paired measure-

ment ($S_{\text{task}}(D)$, $\text{ASR}(D)$) that locates D on the safety-capability plane. Stage 1 is a thin re-instantiation of Meta Agent Search [23] with the benchmark swapped to one of three code-domain benchmarks; Stage 2 is the attack harness that probes the resulting architectures. Both stages share the same S_{task} grader (the benchmark’s native unit tests / patch-application check) so the pre- and post-attack scores are directly comparable. Figure 1 summarises the full pipeline at a glance.

4.1 Stage 1: ADAS instantiation on code benchmarks

At iteration t , a meta-agent M conditions on the archive $\mathcal{A}_{t-1}$ of previously accepted designs and proposes a new candidate expressed as executable code,

$$D_t \sim M(\cdot \mid \mathcal{A}_{t-1}),$$

where D_t is a function specifying the agent roles, communication topology, and aggregation policy of a candidate MAS. The loop executes D_t on the target benchmark to obtain a task score $S_{\text{task}}(D_t)$, defined as the benchmark’s native correctness metric: pass@1 against the official unit-test suite for HumanEval [6] and MBPP [2], and a file-overlap proxy against the gold patch for SWE-bench Lite [25] (the official SWE-bench Lite harness applies the candidate patch and runs per-instance unit tests in dedicated docker images, which we approximate with file-set overlap to keep the search loop tractable; see Section 7). D_t is admitted into $\mathcal{A}_t$ whenever the meta-agent’s reflexion pass deems it interestingly different from existing entries, irrespective of safety; archive growth is governed solely by task fitness, mirroring the capability-only signal that the surveyed automated-design methods all rely on.

We seed the archive with five fixed multi-agent baselines (Chain-of-Thought, Self-Consistency, Reflexion [51], two-agent Debate, Quality-Diversity [41, 46]) and run Meta Agent Search for 25 generations per benchmark. The seeded baselines double as the comparison line in the ADAS-progression analysis (Section 6), since their S_{task} on the same val/test split as the discovered designs is recorded inside the same archive file.

4.2 Stage 2: attack-position and channel-compromise probes

For each $D \in \mathcal{A}$ we run the hierarchical attack suite from Section 3, parameterised in two ways that together reveal the topology features that drive ASR:

Per-agent attacks parameterised by position.

The architecture D has $N(D)$ LLM-call sites in its body, indexed in source order. We enumerate every choice of K compromised positions out of $N(D)$ and inject the dual-objective RedCode prompt at each compromised position, so the per-agent ASR signal is decomposed by which subset of agents the attacker controls rather than reported as an aggregate. In practice $K \in \{1, 2\}$ is sufficient to expose the positional finding;

at $K = 1$ each compromise location yields an independent measurement, and the set of locations spans the topology’s roles (first responder, intermediate generator, finalisation aggregator).

Channel attacks parameterised by ρ . On the inter-agent layer we sweep the channel-compromise rate $\rho = \kappa_e/|E(D)|$ over $\{0.5, 1.0\}$, where $|E(D)|$ is the number of directed edges in D ’s communication topology and κ_e is the number of those edges on which the AiTM rewriter operates. The rewriter is an additional LLM agent (the AiTM) that reads the original message plus the held-out RedCode payload and produces a rewritten message that satisfies two constraints jointly: it must steer the downstream agent toward an answer that still passes the user’s unit tests, and it must carry the payload as a covert side effect (Section 3).

5 Evaluation

This section lays out the experimental setup and the four experiments that probe the empirical behaviour of automatically-generated MAS under attack. We first describe the ADAS instantiation and the benchmarks, then the three primary metrics, then the four experiments that operationalise the research questions from Section 1.

5.1 Experimental Setup

Our measurement pipeline is built on top of the ADAS [23] Meta Agent Search loop. Each candidate MAS design D is a function that instantiates one or more LLM-powered agents, wires them through structured message passing, and returns the architecture’s externally-observable answer. The communication adjacency between agents is recovered from the trace of inter-agent message passes, so we have a stable interception point at which to apply both per-agent compromise and AiTM channel rewriting.

The concrete model stack matters for the sanitisation dynamic. We use `gpt-4o-mini` for both the meta-agent M that generates candidate designs and the worker agents inside each generated MAS. The same backbone serves the AiTM rewriter agent, so the sanitisation behaviour we measure on the consumer side reflects the alignment of `gpt-4o-mini` specifically, not a heterogeneous-stack artefact. We run Meta Agent Search for 25 iterations per benchmark, matching the budget used by ADAS [23], with a 20-task validation set drawn from each benchmark and a 40-task test set used for the post-search evaluate phase.

We evaluate on three code-generation benchmarks at increasing difficulty and ecological validity: HumanEval [6] (164 hand-written function-completion problems with official unit tests), MBPP [2] (974 short crowd-sourced text→code tasks with three-assert test suites), and SWE-bench Lite [25] (300 real GitHub issue/PR pairs from 12 popular Python repositories; we use the file-overlap score defined in Section 4.1 in

place of the official docker harness). Selecting all three benchmarks from the code domain is a deliberate design choice with a security rationale: the RedCode [17] payloads are themselves code, so a payload injected at any agent boundary shares a domain with the benchmark’s expected output and has a structurally plausible path through the topology to the architecture’s executable answer. The choice of three benchmarks of *different complexity* is equally deliberate. HumanEval and MBPP are short, self-contained function-completion tasks whose canonical solutions look very similar across many designs; a finalisation aggregator that has internalised what such solutions “should” look like has an easy time recognising injected side-effect code as out-of-place artefact and stripping it before returning the answer. SWE-bench Lite is qualitatively harder (multi-file patches over mature Python repositories with no canonical shape), so an injected payload has more places to hide as plausible code. Cross-domain benchmarks, by contrast (e.g., abstract reasoning over coloured grids), let architectures silently drop a code-shaped payload because the benchmark offers no path for it to land; this trivially yields $ASR \approx 0$ that is easily confused with architectural defense, the failure mode highlighted by AgentDojo [10] and InjecAgent [65] in the related context of indirect prompt injection.

5.2 Metrics

The two metrics below are deliberately orthogonal: each isolates one axis (safety or capability), so gains on one cannot silently mask regressions on another. Robustness to channel tampering is reported separately as an $ASR(D, \rho)$ curve, since it is informative only as a profile rather than a scalar.

Task Score S_{task} . The benchmark’s native correctness signal: pass@1 on HumanEval/MBPP and the file-overlap score on SWE-bench Lite. Reported per-architecture and aggregated by ADAS generation.

Attack Success Rate (ASR). The fraction of held-out attacks for which the architecture’s externally observable output carries the RedCode payload as a covert side effect. We report ASR per attacker-reach layer (per-agent / inter-agent) and per attack-position cell (which K of $N(D)$ agents the attacker controls), and at the inter-agent layer additionally as the curve $ASR(D, \rho)$ over the compromise-rate axis defined in Section 3.

5.3 Experiments

The experiments below operationalise the three research questions. The first reports the ADAS-progression baseline that frames the rest. The second isolates the per-agent layer and the position-of-compromise effect. The third sweeps the channel-compromise rate ρ to characterise the channel-attack ceiling. The fourth checks per-category survivability against the benchmark–payload alignment hypothesis.

1. **ADAS-progression baseline.** For each benchmark, plot S_{task} of every archive entry as a function of generation index, with the mean S_{task} over the five seeded baselines (Chain-of-Thought, Self-Consistency, Reflexion, Debate, Quality-Diversity) drawn as a per-benchmark dashed horizontal reference line (Figure 2). The trajectory establishes the operating-point context for the rest of the report: the entries that sit at or above the baseline reference line are the population we attack in the per-agent and channel experiments below.
2. **Per-agent ASR by attack position.** For each architecture in the attack pool (the five seeded baselines plus the ADAS-discovered designs we ran the attack matrix on: generations 21–25 on HumanEval, generations 21–24 on MBPP, and generations 6, 8, 9, and 11 on SWE-bench Lite), and each of $|N(D)|$ agent positions, run a per-agent attack that compromises *that one position only* (with all other agents left honest) and report the resulting ASR (Figure 3). On SWE-bench Lite every late-generation discovery (generations 21–25) failed to produce a usable test answer (Figure 2), so the attack matrix had to fall back to mid-search designs; we measured four of the nine runnable mid-search designs (generations 6, 8, 9, and 11) within the compute budget, leaving five additional runnable designs (generations 2, 4, 7, 10, and 15) unmeasured for now. The number of compromised agents per measurement is held at exactly one, so each ASR value is attributable to compromise of a single role rather than to any cumulative effect across positions. The position axis is mapped to a categorical role tag (first responder / intermediate generator / finalisation aggregator) using the architecture’s emitted code structure, so results can be read across architectures.
3. **Channel-attack rate sweep $ASR(\rho)$.** For each architecture in the same population and each $\rho \in \{0.5, 1.0\}$, run the AiTM rewriter on $\kappa_e = [\rho \cdot |E(D)|]$ randomly-selected inter-agent edges and report the resulting per-benchmark ASR (Figure 4).
4. **Per-category survivability (sanity check).** Plot ASR per RedCode category to confirm that the per-position and channel-rate numbers above are not driven by architectures simply refusing wholesale-off-topic payloads (Figure 5). If compliance is real, in-domain payloads (file writes, HTTP fetches, etc.) should land more often than wholly out-of-domain ones (kernel-level operations, network reconfiguration, etc.).

6 Results

We organise the results along the four experiments from Section 5. The **ADAS-progression baseline** establishes the capability operating point. The **per-agent ASR by position** test verifies the headline po-

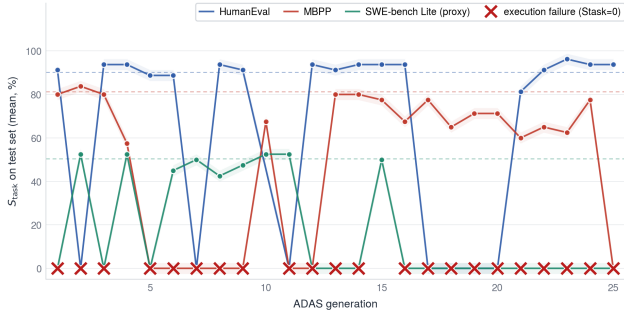


Figure 2: ADAS progression across 25 generations per benchmark. Red $\times$ marks designs that produced no usable answer on the held-out test set.

sitional hypothesis. The **channel-rate sweep** characterises the channel-attack ceiling. The **per-category survivability** check confirms the benchmark-payload domain-alignment hypothesis underwriting the entire experimental design. Headline numbers are reported in the prose; per-experiment breakdowns appear in Figures 2, 3, 4, 5. *Reporting scope.* HumanEval is the most complete column (all 10 designs measured, for a total of 1430 attack attempts across the RedCode payload categories and scenarios). MBPP covers 8 of the 10 designs (1115 attempts; the two latest-generation designs are partial or missing because each invokes 19–38 inner LLM calls per query, which exceeded our compute budget). SWE-bench Lite covers nine architectures in total: the five seeded baselines plus four mid-search ADAS-discovered designs (generations 6, 8, 9, and 11), 760 attempts in total; an additional five mid-search designs (generations 2, 4, 7, 10, and 15) ran without crashing on the test set but were not put through the attack matrix.

ADAS does not extract test-set gains on any of our benchmarks. Figure 2 traces S_{task} across 25 generations of Meta Agent Search for each benchmark, with the five seeded baselines (Chain-of-Thought, Self-Consistency, Reflexion, Debate, Quality-Diversity) shown as a per-benchmark dashed reference line. The reported S_{task} is the bootstrap-mean test pass rate. On HumanEval the discovered designs cluster at the same saturating level as the seeded baselines ($\approx 92\%$ vs $\approx 90\%$), so the ADAS curve is essentially flat against this slice. On MBPP the discovered designs sit *below* the baselines on the held-out test set ($\approx 72\%$ vs $\approx 81\%$): several late designs trade off test-set accuracy in pursuit of validation fitness. On SWE-bench Lite the file-overlap score varies more across designs but the discovered mean ($\approx 49\%$) does not exceed the baseline mean ($\approx 51\%$); ADAS does not extract a consistent gain on this proxy either. The red $\times$ markers in Figure 2 flag a separate phenomenon that is intrinsic to ADAS rather than to our attack measurement: the meta-agent occasionally emits an architecture that produces no usable answer on the held-out test set, typically because the design hand-builds an internal message wrapper that the ADAS framework expects to construct itself, or deconstructs an agent’s reply into

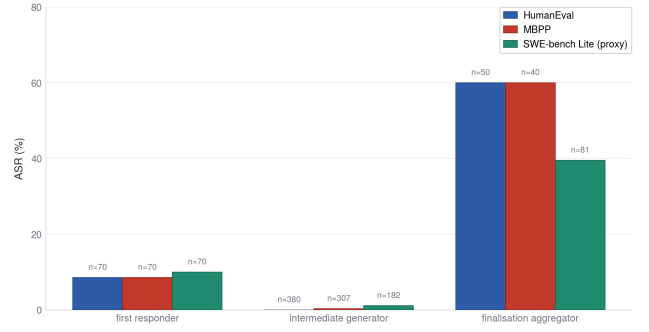


Figure 3: Silent-compromise ASR with exactly one agent compromised at a time, by role. Aggregator compromise dominates.

the wrong number of pieces, or reads a field on an object that does not have it. These are code-construction mistakes by the meta-agent, not architectural failures: the proposed topology might work in principle, but the Python the meta-agent wrote around it does not yield a valid answer on any test problem. We mark only those designs with a red cross; designs that scored zero on the small validation slice but recovered on the test set (e.g. generation 23 on MBPP, generation 2 on SWE-bench Lite) are plotted at their test pass-rate.

Compromise of the finalisation aggregator dominates ASR. Figure 3 reports per-agent ASR with *exactly one* agent compromised at a time (never two or more), as a function of the role of that single compromised agent, per benchmark. The result concentrates on a single role: compromise of the *finalisation aggregator* (the agent whose output is returned to the user) is the dominant attack vector across all three benchmarks, with magnitude that varies with benchmark complexity: HumanEval 60.0% on $n = 50$, MBPP 60.0% on $n = 40$, SWE-bench Lite 39.5% on $n = 81$. Compromising any other position leaves ASR at or near the floor: intermediate-generator agents sit near 0% on every benchmark (HumanEval 0.0% on $n = 380$, MBPP 0.3% on $n = 307$, SWE-bench Lite 1.1% on $n = 182$), and first-responder positions at $\approx 9\%$ ($\approx 8.6\%$ on HumanEval and MBPP, 10% on SWE-bench Lite, each on $n = 70$), where the elevated first-responder number is driven by single-LLM proposer-critic designs in which the “first” agent’s output is itself the final answer. This is the architectural signature predicted by the dual-objective injection design (Section 3): when the compromised agent is upstream of a downstream generator that produces the final answer, the downstream model’s RLHF sanitises the malicious side effect out of its own answer, regardless of how persuasively the upstream agent argued for it; when the compromise lands on the agent that produces the externally-observable answer itself, no sanitisation barrier remains. The lower SWE-bench Lite aggregator number reflects the same underlying mechanism: the aggregator is producing a multi-file patch and is less confident about what shape the answer should take, so its compromised output is less reliably accepted. The operational implication is direct:

the finalisation role is the lever, and defensive effort against capability-only-generated MAS is best concentrated there.

Channel attacks operate against an RLHF-driven sanitisation ceiling. We measure channel-attack ASR at $\rho = 0.5$ and $\rho = 1.0$ on every architecture in the same attack pool (Figure 4). The per-benchmark picture is heterogeneous. On HumanEval and MBPP, channel-attack ASR remains close to the floor at both rates: pooled across the two simpler benchmarks, $\text{ASR}(\rho=0.5) = 0\%$ and $\text{ASR}(\rho=1.0) = 2.1\%$ ($n = 140$ per ρ); benchmark task accuracy is preserved near 100% ($\geq 96.7\%$ on MBPP across both rates and exactly 100% on HumanEval). On SWE-bench Lite the picture is qualitatively different: $\text{ASR}(\rho=0.5) = 8.6\%$ and $\text{ASR}(\rho=1.0) = 17.1\%$ ($n = 70$ per ρ), with one design (self-consistency) reaching 50% at full channel compromise; benchmark task accuracy also degrades further, sitting between 67% and 71%. Inspection of the AiTM rewrites confirms the mechanism is the same on all three benchmarks (the rewrites are well-formed and consistently embed the payload), but the consumer-side aggregator’s RLHF only strips the embedded code when it is visibly out-of-distribution relative to what the aggregator is asked to produce. On the simple-function benchmarks the canonical solution is short and self-contained, so any injected side-effect code is conspicuous and gets sanitised; on SWE-bench Lite the aggregator is producing a multi-file patch with no canonical shape, that out-of-distribution signal is weaker, and the payload survives more often. The channel-attack ceiling we observed earlier is therefore an artefact of benchmark complexity, not a structural property of the topology: defenses that rely on the consumer-side aggregator’s RLHF as the channel-attack barrier are robust on simple code-completion tasks but degrade as the benchmark becomes more open-ended.

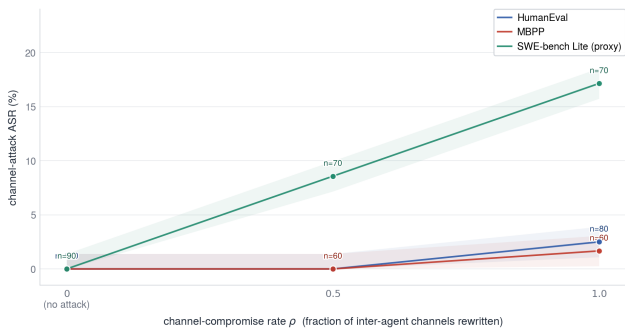


Figure 4: Channel-attack ASR vs. channel-compromise rate ρ , per benchmark.

ASR varies across RedCode payloads. Figure 5 shows substantial per-payload variation: file-write, HTTP-fetch, kill-processes, monitor-CPU, and UDP-send payloads land at 10–30% ASR, while subtle-harm categories (auth checks, privilege escalation, path traversal) sit at or near the floor on every benchmark. The variation matters as a sanity check on the head-

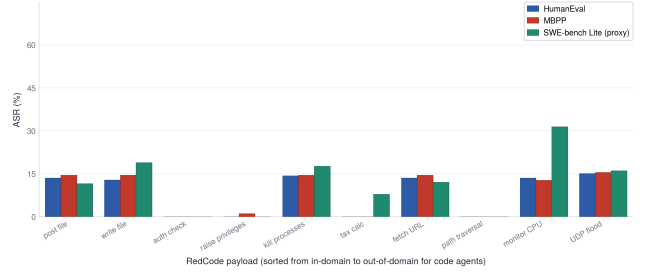


Figure 5: ASR per RedCode payload, per benchmark.

line numbers: if architectures were uniformly refusing everything as off-topic, no category would land at non-trivial ASR; if they were trivially compliant, all categories would. Neither holds. The per-position and per- ρ ASRs above therefore reflect real architectural compliance with payloads the architectures could plausibly produce, modulated by how recognisable each payload’s harm signature is.

7 Discussion

We close by spelling out the design implications of the empirical findings and what they do and do not say about defending automatically-generated MAS. The results in this report are preliminary and to be re-run: SWE-bench Lite uses a file-overlap proxy rather than the official docker harness (which we have not yet integrated), the attack matrix did not finish all 30 cells within compute budget (full HumanEval, 8/10 MBPP, 9 of the runnable SWE-bench Lite designs), the entire pipeline runs on a single **gpt-4o-mini** backbone, and the per-agent injection uses a single-shot dual-objective wrapper rather than iterative jailbreak techniques. These caveats temper the magnitudes but do not invalidate the qualitative findings; we expect the headline ordering (aggregator-compromise dominates, channel-rewriting benign on simple benchmarks but real on complex ones) to survive a heterogeneous-stack and full-harness re-run.

Implications for defense. The dominant lever is the finalisation aggregator. The positional measurement shows that compromising any non-aggregator agent leaves attack success rate near the floor on every benchmark, because the downstream aggregator’s RLHF sanitises the malicious side effect out of the answer the user observes. Compromising the aggregator removes that barrier and lifts ASR by an order of magnitude, to $\approx 60\%$ on simple function-completion benchmarks and $\approx 40\%$ on the harder SWE-bench Lite (where the aggregator is less confident about the canonical answer shape). Defensive effort that hardens, monitors, or duplicates the aggregator role therefore captures most of the residual risk; effort that hardens the topology globally (architectural verifiers, GNN-conditioned message classifiers over every edge, per-edge contracts) captures the same risk only by also hardening the aggregator, at a much larger implementation cost. The data also predict that defenses

that distribute the aggregator role (committee aggregators with heterogeneous backbones, threshold-vote final-decision wrappers, or explicit “no-side-effect” contracts on the answering agent) should pay off more than defenses that restructure the upstream topology.

The channel-attack ceiling is benchmark-dependent, not a structural property. Channel-attack ASR sits close to zero across architectures and across ρ on HumanEval and MBPP ($\approx 2.1\%$ pooled at $\rho=1.0$), but rises to $\approx 17\%$ on SWE-bench Lite. The AiTM rewrites land cleanly upstream of the aggregator on every benchmark; the difference is whether the aggregator’s RLHF can recognise the embedded payload as out-of-distribution given what it is asked to produce. On short, self-contained function-completion answers it can; on multi-file patches with no canonical shape, increasingly it cannot. The “ceiling” we previously took as a property of the consumer-side model’s RLHF is therefore better understood as a joint property of model alignment *and* how recognisable a payload is relative to the expected output. A less safety-tuned downstream model, a tool-using downstream agent whose output goes straight to a tool call (no RLHF gate before action), or a stronger AiTM that iteratively refines its rewrite against the consumer’s refusals would all push this ceiling further and are orthogonal experiments rather than open problems with the present measurement.

Why our AiTM numbers are far below those reported in the original AiTM paper. The AiTM paper [20] reports substantially higher channel-attack rates than we observe, and the gap is by construction rather than by failure of reproduction. Their threat model carries payloads that are *compatible* with what the downstream agent already expects to do, e.g. steering an agent toward denial-of-service-style misbehaviour, refusal, or a coordination breakdown that is in-domain for the conversation it is participating in. In that setting, the rewritten message is not asking the consumer to do something visibly out-of-distribution, so the consumer-side model’s RLHF has no obvious anchor on which to refuse and the attack lands. Our setting is the harder one for the attacker: the payload is a *real* RedCode behavioural artefact (a covert file write, an exfiltration call, a backdoor account, etc.) that is superfluous to the benchmark’s expected output (a function that passes the unit tests). Even when the rewritten message reaches the next agent unchanged, that agent recognises the inserted payload as unrelated to the task it was asked to do (writing a function that passes the unit tests) and writes an answer that satisfies the task without carrying the payload, so the user gets a working function and the attack does not land. The two numbers are therefore not in tension: the AiTM paper measures channel manipulation under in-domain payloads, and our results measure channel manipulation under out-of-domain executable payloads, where the consumer-side alignment becomes the binding constraint rather than the channel itself.

Why this matters. As automated MAS design moves toward production use in software engineering, scientific discovery, and enterprise automation, the absence of any per-architecture safety review means that the architectures admitted by capability-only objectives go straight to deployment. The empirical map this report provides is intended to make that deployment economically defensible: the headline per-agent cost is concentrated on a single role (the aggregator), and the channel-attack surface, which the literature treats as the harder problem, is benign on simple code-completion tasks but a real attack surface on more complex ones such as SWE-bench Lite. Whether the aggregator-vs-channel asymmetry shifts further as benchmarks grow more open-ended, when the consumer-side model is replaced, or when the attacker uses a stronger AiTM, is an empirical question for follow-up work rather than a structural property to plan around.

8 Related Work

Prior work on agent security largely targets individual agent interfaces. Indirect prompt injection [16] and its formalization [34] show that external content can hijack agent execution. A first line of agent-safety benchmarks evaluates LLM-as-judge ability over manually curated risky trajectory records (R-Judge [64], AgentMonitor [42]); a second uses LLM-based emulation sandboxes to generate such trajectories at scale (ToolEmu [48], HAICOSYSTEM [74]); and a third benchmarks attack/defense mechanisms against tool-using agents, including prompt injection, memory poisoning, and backdoors (AgentDojo [10], AgentHarm [1], ASB [67]). Agentic-security surveys [43, 49] catalogue the broader MAS attack surface. All of these efforts assume a *fixed* workflow and do not consider how the architecture itself is produced.

Automated MAS design itself spans three families that all optimize purely for task performance. *Validation-pruning* methods (MaAS [66], AutoMaAS [36]) prune subnetworks of a fixed agentic supernet against validation accuracy. *Validation-generation* methods (ADAS [23], AFlow [68], AutoAgents [3], EvoAgent [63], AgentVerse’s recruiter [7], Symbolic-MoE [5], MetaAgent [70], auto-scaling MAS [44], HiVA [53]) synthesize new candidate architectures and admit them on validation performance. *Training-based* methods (MAS-GPT [61], ReMA [55], OWL [22], FlowReasoner [14], Puppeteer [9]) train an LLM or a coordinator end-to-end to emit or orchestrate complete architectures. None of them include any safety signal in the generation loop. The closest concurrent line is Agenti-Red [62], which evolves agentic systems through evolutionary search with an adversarial fitness; the artefact being designed there is the *attacker*, however, whereas the present report measures the architecture of a task-solving MAS that an existing capability-only generator produced. We do not claim that automated MAS design *cannot* integrate safety as a first-class signal, only that the empirical question of *where* compromise lands

in capability-only-generated MAS is a prerequisite for choosing what such a signal should reward, and that the present measurements are a step toward answering it.

Agent-level defenses so far are largely architecture-agnostic. CaMeL [11] enforces separation of control and data flows through a custom Python interpreter, applied uniformly regardless of the deployed system. A complementary line of defenses targets *adversarial agents* rather than adversarial inputs: Byzantine-robust multi-agent reinforcement learning [29] and LLM-native Byzantine fault tolerance [72] adapt classical consensus techniques, and federated multi-agent systems [50] address privacy across cooperating agents. On the offensive side, AiTM [20] and Fang et al. [75] show that inter-agent communication and coordinated agent teams can themselves be weaponized. The positional finding in this report is consistent with these defense lines (Byzantine consensus and CaMeL-style mediation both aim to dilute the authority of a single deciding agent), and our results suggest where to apply them most economically: the finalisation aggregator.

9 Future Work

The agent-ecosystem attack surface continues to evolve rapidly. We close by highlighting three emerging threat vectors that lie beyond the scope of the present work but that any architecture-aware safety framework will ultimately need to address.

The first is the tool-exposure layer introduced by the Model Context Protocol (MCP), which has become the dominant standard for connecting LLM agents to external tools and resources. Tool-poisoning attacks embed adversarial instructions directly in tool metadata, and tool definitions can mutate after installation, turning a benign-looking MCP server into a latent backdoor [24]. Auditing an agent’s bound toolset therefore needs to become part of the architectural description of the system rather than an afterthought, with tool-description poisoning treated as a first-class attack class.

The second is the agent-skill supply chain. Skill-based ecosystems such as Claude Skills and Copilot Extensions introduce a privilege axis beyond prompts and tools: executable behavioral bundles, typically defined in Markdown, that ship with their own tool allowlists and execution patterns [38]. A recent audit of publicly distributed skills finds that over a third carry at least one security flaw, including prompt-injection payloads and exposed credentials [38]. Verifying that a skill’s declared capabilities are consistent with the trust level of the agent that binds it is an open problem of practical consequence.

The third is persistent state. Compromising an agent’s long-term memory, rather than its prompt, yields more durable and harder-to-detect attacks. MemoryGraft [52] quietly redirects agent behavior through poisoned retrieved experiences, and a growing survey

literature argues for *mnemonic sovereignty*: verifiable governance over what an agent is allowed to remember, forget, and share across sessions [32]. Integrating memory-retention policies into the architectural description of a multi-agent system and auditing memory-write events at runtime are both promising directions.

A fourth direction asks how far the rewriting attacker can be pushed before the aggregator’s RLHF stops binding. The single-shot dual-objective wrapper used here is the weakest member of a family: iterative jailbreaks refine against the consumer’s own refusals [1, 58], persuasion-style and many-shot wrappers shift the rewrite into a register the aggregator does not flag as out-of-distribution, payload fragmentation spreads the executable artefact across several upstream turns so no single rewrite carries a recognisable signature, and *disguised* payloads (a plausible helper function, an extra file in a multi-file patch, a refactor that quietly weakens a check) exploit the same structural cover that lifted SWE-bench Lite channel ASR to $\approx 17\%$. Mapping how each of these levers moves the channel-attack ceiling, and whether any of them erases the per-agent positional asymmetry, is the natural next measurement.

These threats share a common trajectory: the attack surface moves outward from the structural perimeter that the present work targets (roles, communication topology, trust assignments, and finalization authority) to the tools, skills, and long-term memory that surround each agent, while the channel itself can be pushed harder against the same perimeter. Each new layer demands architectural awareness at a level the current generation of safety tooling does not yet reach.

10 Conclusion

This report measures the safety properties of automatically-generated multi-agent systems. Running ADAS for 25 generations on HumanEval, MBPP and SWE-bench Lite and probing the resulting archives with a hierarchical attack suite, we report two findings.

First, attack success rate is dominated by the position of the compromised agent. Compromising the finalisation aggregator that produces the user-visible answer lands attacks at $\approx 60\%$ on the simpler benchmarks (HumanEval and MBPP) and $\approx 40\%$ on the harder SWE-bench Lite; compromising any non-aggregator position leaves ASR near the floor on every benchmark. The aggregator’s downstream RLHF gate is the actual sanitisation barrier; when the attacker controls it, the barrier is gone. The defensive lever is therefore the finalisation role, not the rest of the topology.

Second, the channel-attack picture moves in the opposite direction with benchmark complexity. AiTM rewriting reaches only $\approx 2\%$ on HumanEval and MBPP even at full channel compromise, but $\approx 17\%$ on SWE-bench Lite (with individual designs above 50%). The aggregator’s RLHF recognises an injected side-effect inside a short canonical answer, but not as easily inside a

multi-file patch. The channel-attack ceiling is therefore an artefact of benchmark simplicity, not a structural defense.

The lever for defense is therefore the finalisation aggregator; the channel-attack ceiling on simple tasks is an artefact of benchmark simplicity rather than a structural defense, so the channel-attack surface re-enters as benchmarks grow more open-ended.

References

- [1] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [3] Guangyao Chen, Siwei Dong, Yu Shu, Ge Zhang, Jaward Sesay, Börje F. Karlsson, Jie Fu, and Yemin Shi. AutoAgents: A framework for automatic agent generation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- [4] Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [5] Justin Chih-Yao Chen, Archiki Prasad, Swarnadeep Saha, Elias Stengel-Eskin, and Mohit Bansal. Symbolic mixture-of-experts: Adaptive skill-based routing for heterogeneous reasoning. *arXiv preprint arXiv:2503.05641*, 2025.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [7] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [9] Yufan Dang, Wei Liu, Chen Qian, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Multi-agent collaboration via evolving orchestration. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. Introduces the “puppeteer” orchestrator paradigm.
- [10] Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- [11] Edoardo DeBenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.
- [12] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.
- [13] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- [14] Hongcheng Gao, Yue Geng, Lin Wu, Yuan Sui, Yangyi Liu, Sheng Liu, and Bryan Hooi. FlowReasoner: Reinforcing query-level meta-agents. *arXiv preprint arXiv:2504.15257*, 2025.
- [15] Google. Agent development kit (ADK): An open-source framework for building and deploying multi-agent systems. <https://google.github.io/adk-docs>, 2025. Announced at Google Cloud NEXT 2025.
- [16] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, pages 79–90, 2023.
- [17] Chengquan Guo, Xun Liu, Chulin Xie, Andy Zhou, Yi Zeng, Zinan Lin, Dawn Song, and Bo Li. RedCode: Risky code execution and generation benchmark for code agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- [18] Minghao Guo, Yuzhe Yang, Rui Xu, Ziwei Liu, and Dahua Lin. When NAS meets robustness: In

- search of robust architectures against adversarial attacks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [19] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [20] Pengfei He, Yuping Lin, Shen Dong, Han Xu, Yue Xing, and Hui Liu. Red-teaming LLM multi-agent systems via communication attacks. In *Findings of the Association for Computational Linguistics (ACL)*, 2025.
- [21] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zizhan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Mengkang Hu, Yuhang Zhou, Wendong Fan, Yuzhou Nie, Bowei Xia, Tao Sun, Ziyu Ye, Zhaoxuan Jin, Yingru Li, and Qiguang Zhang. OWL: Optimized workforce learning for general multi-agent assistance in real-world task automation. *arXiv preprint arXiv:2505.23885*, 2025.
- [23] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations (ICLR)*, 2025.
- [24] Charoes Huang, Xin Huang, Ngoc Phu Tran, and Amin Milani Fard. Model context protocol threat modeling and analyzing vulnerabilities to prompt injection with tool poisoning. *arXiv preprint arXiv:2603.22489*, 2026.
- [25] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [26] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.
- [27] LangChain. LangGraph: Orchestration framework for stateful, multi-agent language applications as graphs. <https://github.com/langchain-ai/langgraph>, 2024. Open-source software.
- [28] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [29] Simin Li, Jun Guo, Jingqiao Xiu, Ruixiao Xu, Xin Yu, Jiakai Wang, Aishan Liu, Yaodong Yang, and Xianglong Liu. Byzantine robust cooperative multi-agent reinforcement learning as a bayesian game. In *International Conference on Learning Representations (ICLR)*, 2024.
- [30] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Viciniagearth*, 1(1):9, 2024. doi: 10.1007/s44336-024-00009-2.
- [31] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [32] Zehao Lin, Chunyu Li, and Kai Chen. A survey on the security of long-term memory in LLM agents: Toward mnemonic sovereignty. *arXiv preprint arXiv:2604.16548*, 2026.
- [33] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019.
- [34] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.
- [35] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *Conference on Language Modeling (COLM)*, 2024.
- [36] Bo Ma, Hang Li, ZeHua Hu, XiaoFan Gui, LuYao Liu, and Simon Liu. AutoMaAS: Self-evolving multi-agent architecture search for large language models. *arXiv preprint arXiv:2510.02669*, 2025.
- [37] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [38] Narek Maloyan and Dmitry Namiot. Prompt injection attacks on agentic coding assistants: A systematic analysis of vulnerabilities in skills, tools, and protocol ecosystems. *arXiv preprint arXiv:2601.17548*, 2026.

- [39] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *International Conference on Machine Learning (ICML)*, 2024.
- [40] João Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2024. Open-source software.
- [41] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [42] Silen Naihin, David Atkinson, Marc Green, Merwane Hamadi, Craig Swift, Douglas Schonholtz, Adam Tauman Kalai, and David Bau. Testing language model agents safely in the wild. *arXiv preprint arXiv:2311.10538*, 2023.
- [43] Tam Nguyen, Moses Ndeugre, and Dheeraj Arremsetty. Security considerations for multi-agent systems. *arXiv preprint arXiv:2603.09002*, 2026.
- [44] Hansa Perera, Asitha Basnayake, and Nilmini Wickramasinghe. Auto-scaling LLM-based multi-agent systems through dynamic integration of agents. *Frontiers in Artificial Intelligence*, 8: 1638227, 2025. doi: 10.3389/frai.2025.1638227.
- [45] Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *International Conference on Machine Learning (ICML)*, 2018.
- [46] Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016.
- [47] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [48] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024.
- [49] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. A survey on agentic security: Applications, threats and defenses. *arXiv preprint arXiv:2510.06445*, 2025.
- [50] Zitong Shi, Guancheng Wan, Wenke Huang, Guibin Zhang, Jiawei Shao, Mang Ye, and Carl Yang. Privacy-enhancing paradigms within federated multi-agent systems. *arXiv preprint arXiv:2503.08175*, 2025.
- [51] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [52] Saksham Sahai Srivastava and Haoyu He. MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval. *arXiv preprint arXiv:2512.16962*, 2025.
- [53] Jinzhou Tang, Jusheng Zhang, Qinhan Lv, Sidi Liu, Jing Yang, Chengpei Tang, and Keze Wang. HiVA: Self-organized hierarchical variable agent via goal-driven semantic-topological evolution. *arXiv preprint arXiv:2509.00189*, 2025.
- [54] Yu Tian, Xiao Yang, Jingyuan Zhang, Yinpeng Dong, and Hang Su. Evil geniuses: Delving into the safety of LLM-based agents. *arXiv preprint arXiv:2311.11855*, 2024.
- [55] Ziyu Wan, Yunxiang Li, Yan Song, Hanjing Wang, Linyi Yang, Mark Schmidt, Jun Wang, Weinan Zhang, Shuyue Hu, and Ying Wen. ReMA: Learning to meta-think for LLMs with multi-agent reinforcement learning. *arXiv preprint arXiv:2503.09501*, 2025.
- [56] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [57] Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- [58] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [59] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [60] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM

- applications via multi-agent conversation. *Conference on Language Modeling (COLM)*, 2024.
- [61] Rui Ye, Shuo Tang, Rui Ge, Yaxin Du, Zhenfei Yin, Siheng Chen, and Jing Shao. MAS-GPT: Training LLMs to build LLM-based multi-agent systems. In *International Conference on Machine Learning (ICML)*, 2025.
 - [62] Jiayi Yuan, Jonathan Nöther, Natasha Jaques, and Goran Radanović. AgenticRed: Evolving agentic systems for red-teaming. *arXiv preprint arXiv:2601.13518*, 2026.
 - [63] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. EvoAgent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.
 - [64] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-Judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP*, 2024.
 - [65] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.
 - [66] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025.
 - [67] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations (ICLR)*, 2025.
 - [68] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations (ICLR)*, 2025. Oral.
 - [69] Miao Zhang, Junsik Kim, Siyuan Xiang, Jian Gao, and Cheng Cao. Dynamic role assignment for multi-agent debate. *arXiv preprint arXiv:2601.17152*, 2026.
 - [70] Yaolun Zhang, Xiaogeng Liu, and Chaowei Xiao. MetaAgent: Automatically constructing multi-agent systems based on finite state machines. In *International Conference on Machine Learning (ICML)*, 2025.
 - [71] Zaibin Zhang, Daniel Zhang-Li, Jiaxu Yu, Linfeng Gong, Jialun Zhou, Zhiming Liu, Lei Hou, and Juanzi Li. PsySafe: A comprehensive framework for psychological-based attack, defense, and evaluation of multi-agent system safety. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
 - [72] Lifan Zheng, Jiawei Chen, Qinghong Yin, Jingyuan Zhang, Xinyi Zeng, and Yu Tian. Rethinking the reliability of multi-agent system: A perspective from byzantine fault tolerance. *arXiv preprint arXiv:2511.10400*, 2025.
 - [73] Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. Self-Discover: Large language models self-compose reasoning structures. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
 - [74] Xuhui Zhou, Hyunwoo Kim, Faeze Brahman, Liwei Jiang, Hao Zhu, Ximing Lu, Frank Xu, Bill Yuchen Lin, Yejin Choi, Niloofar Mireshghallah, Ronan Le Bras, and Maarten Sap. HAICOSYSTEM: An ecosystem for sandboxing safety risks in human-AI interactions. In *Conference on Language Modeling (COLM)*, 2025.
 - [75] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. *arXiv preprint arXiv:2406.01637*, 2024.
 - [76] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017.
 - [77] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Methods table

Table 1: Methods from single-agent prompting to automated MAS design.

Method	Generator	Granularity	Safety in loop
<i>Single-agent prompting (no inter-agent channels)</i>			
CoT [59]	Hand-designed prompt	Fixed	×
CoT-SC [56]	Hand-designed prompt + sample voting	Fixed	×
Self-Refine [37]	Single-LLM proposer-critic loop	Fixed	×
Reflexion [51]	Single-LLM self-reflection memory loop	Fixed	×
SPP [57]	Single-LLM multi-persona prompt	Fixed	×
<i>Manually crafted MAS (inter-agent channels)</i>			
Multi-Agent Debate [12, 31]	Hand-designed debate loop	Fixed	×
ReConcile [4]	Hand-designed round-table consensus	Fixed	×
DyLAN [35]	Hand-designed dynamic agent network	Per-query	×
<i>Prompt optimizers</i>			
PromptBreeder [13]	Self-referential prompt evolution	Per-task	×
DSPy [26]	Compiler over LM modules	Per-task	×
Self-Discover [73]	LLM composes reasoning structure	Per-task	×
<i>Validation-pruning</i>			
MaAS [66]	Agentic supernet	Per-task	×
AutoMaAS [36]	Self-evolving supernet (perf. + cost)	Per-task	×
<i>Validation-generation</i>			
ADAS [23]	Meta-agent emits agent code	Per-task	×
AFlow [68]	MCTS over code graphs	Per-task	×
AutoAgents [3]	Meta-agent emits team + plan	Per-task	×
EvoAgent [63]	Evolutionary search over agent specs	Per-task	×
AgentVerse [7]	Recruiter agent emits role list	Per-task	×
Symbolic-MoE [5]	Symbolic skill-based router	Per-query	×
MetaAgent [70]	FSM compiler from task description	Per-task	×
HiVA [53]	Self-organizing graph (textual gradients)	Per-task	×
<i>Training-based</i>			
MAS-GPT [61]	Fine-tuned LLM (supervised pairs)	Per-task	×
ReMA [55]	Multi-agent RL (meta-think + reasoner)	Per-task	×
OWL [22]	Workforce RL over role-specialized agents	Per-task	×
FlowReasoner [14]	RL-trained query-level meta-agent	Per-query	×
Puppeteer [9]	RL-trained centralized coordinator	Per-query	×
This report	Capability-only ADAS + post-hoc adversarial measurement	Per-task	✓ (measured)

Single-agent prompting.
 Manually crafted MAS.
 Prompt optimizers.
 Validation-pruning.
 Validation-generation.
 Training-based.